Object-Oriented Programming II

Last session on OOP…

Model Reality

Map systems to real-world entities.

Define Structure

Classes create the blueprint.

Design Relational Intent

Connect objects purposefully.

Balance

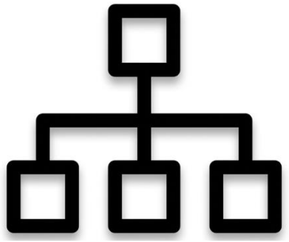
High cohesion, low coupling.

Discipline

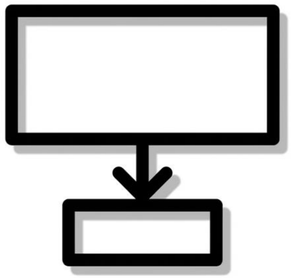
OOP is a framework for thinking.

This session on OOP

Inheritance

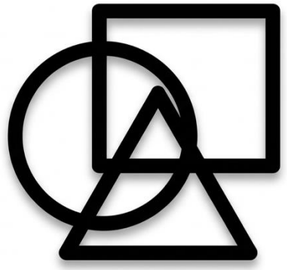


Superclasses & Subclasses

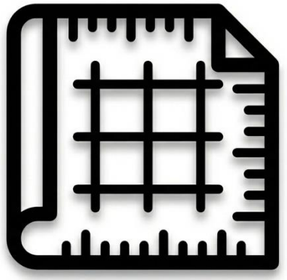


Overrides & Scopes

Polymorphism



Abstract Classes



Case Study

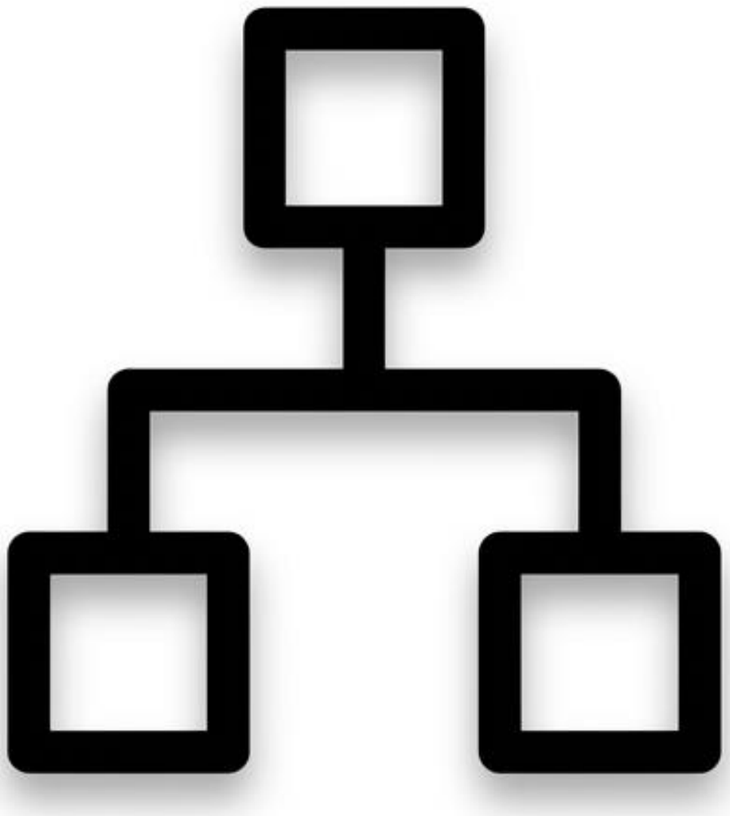
Imagine you’re building software for a Zoo

The zoo contains may animals - Lion, Elephant, Penguin, etc.

All animals share some common characteristics(attributes).

But each animal behaves differently.

Let’s explore modelling this with OOP.

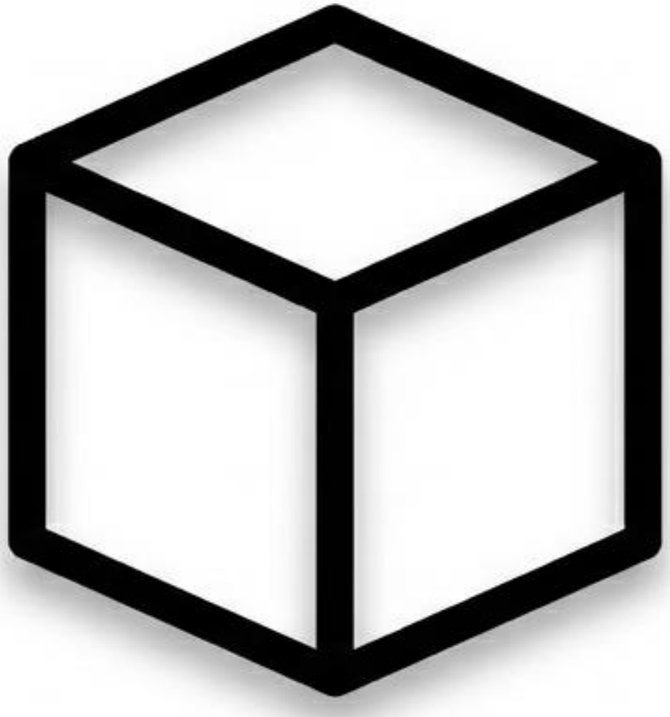


Reusing functionality through inheritance

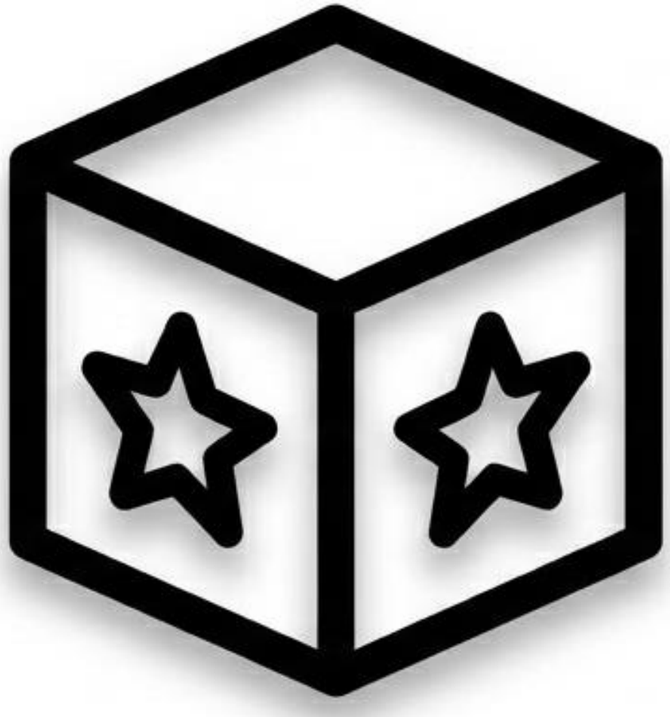
Inheritance allows a child class to inherit attributes and methods from a parent class. The child can also extend or modify this behaviour.

- Reduces code duplication
- Promotes the reuse of common logic
- Creates logical hierarchies between classes

Superclasses define, subclasses specialise



Superclass (Parent Class):
Represents a general concept. Defines common attributes and shared methods.



Subclass (Child Class):
Represents a more specific concept. Extends the parent behaviour and can override methods.

```python
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def eat(self):
        print(self.name, "is eating")

    def sleep(self):
        print(self.name, "is sleeping")


lion = Animal("Simba", 5)

lion.eat()
lion.sleep()
```

```python
from animal import Animal

class Lion(Animal):
    def make_sound(self):
        print(self.name, "roars")

class Elephant(Animal):
    def make_sound(self):
        print(self.name, "trumpets")

lion = Lion("Simba", 5)
elephant = Elephant("Dumba", 10)
```

Controlling data ownership with method scope

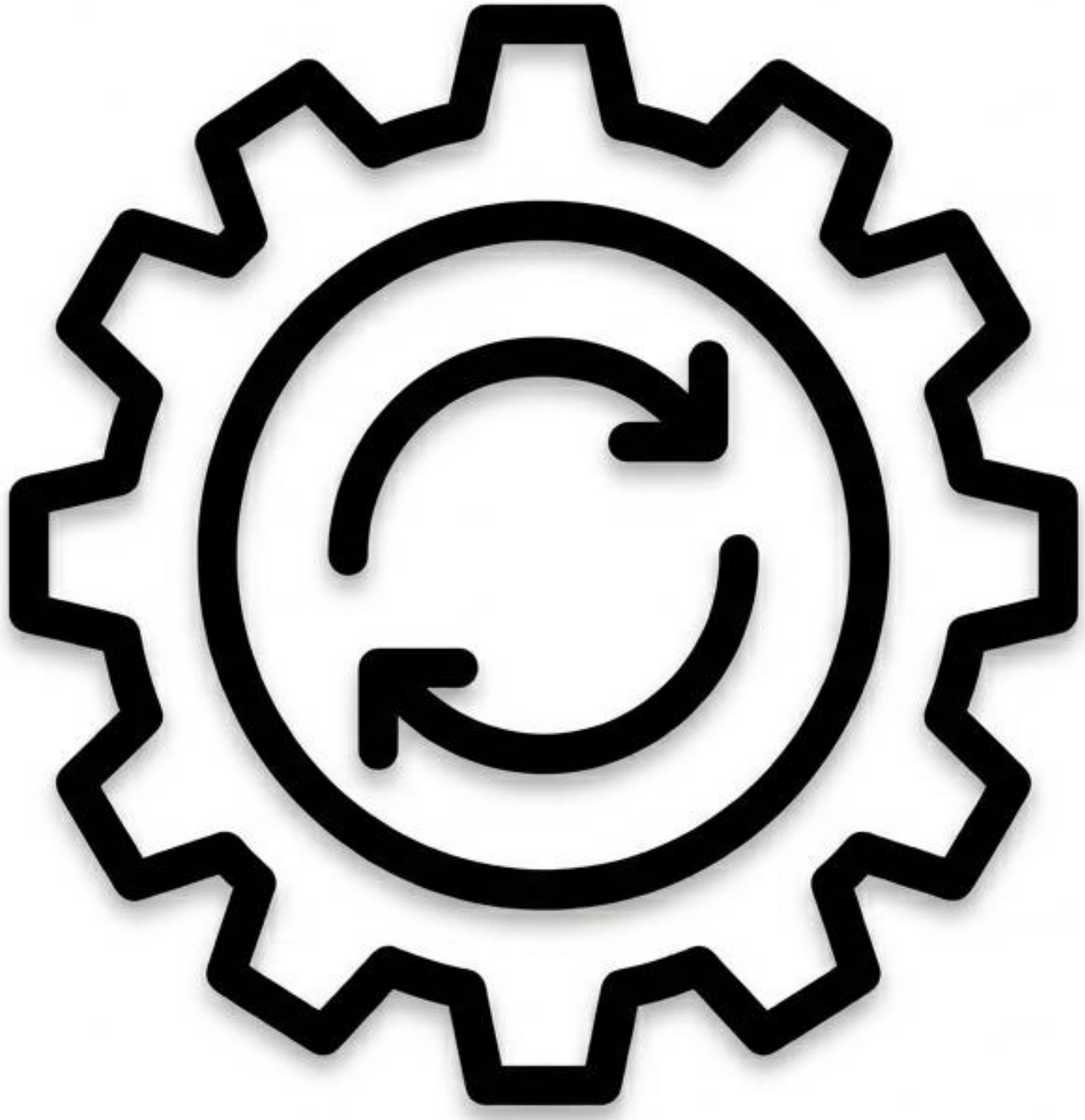
Scope determines where variables and methods are accessible. Understanding scope helps control data ownership.

Instance Scope: Belongs to a specific object. Each object stores its own values (e.g., car1.color = red, car2.color = blue).

Class Scope: Shared across all instances of a class (e.g., Car.wheels = 4 ensures all cars share the same number of wheels).

```python
class Car:

    wheels = 4

    def __init__(self, make, model):
        self.make = make
        self.model = model

car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Civic")

print(car1.make, car1.model, "has", car1.wheels, "wheels")
print(car2.make, car2.model, "has", car2.wheels, "wheels")
```

```python
class Animal:

    kingdom = "Animalia”

    def __init__(self, name):
        self.name = name

lion = Animal("Simba")
elephant = Animal("Dumbo")

print(lion.name)
print(elephant.name)

print(lion.kingdom)
print(elephant.kingdom)
```



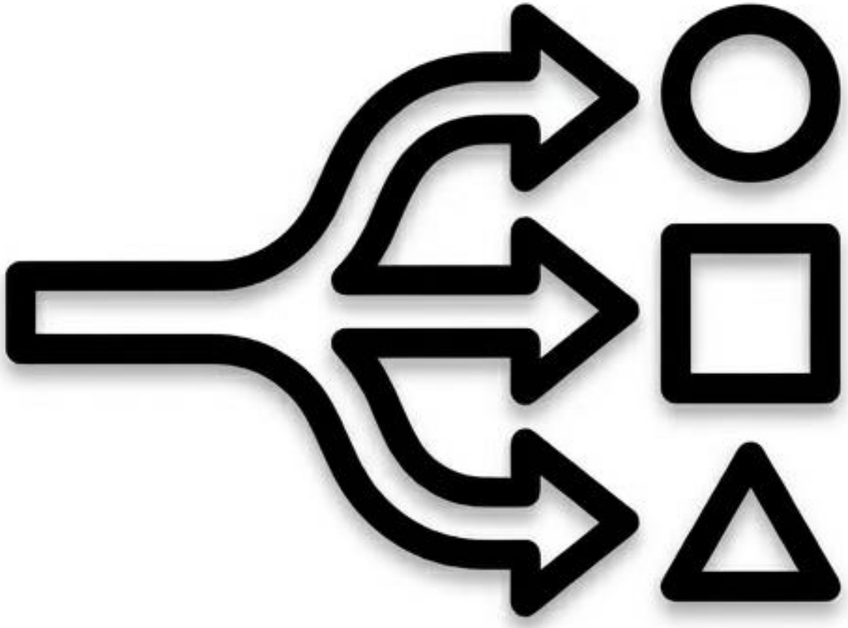
Customising behaviour with method overriding

Method overriding occurs when a subclass provides its own implementation of a method defined in the parent class.
The method name **remains the same**, but the internal behaviour changes.

This allows subclasses to heavily customise their behaviour while strictly maintaining a shared interface.

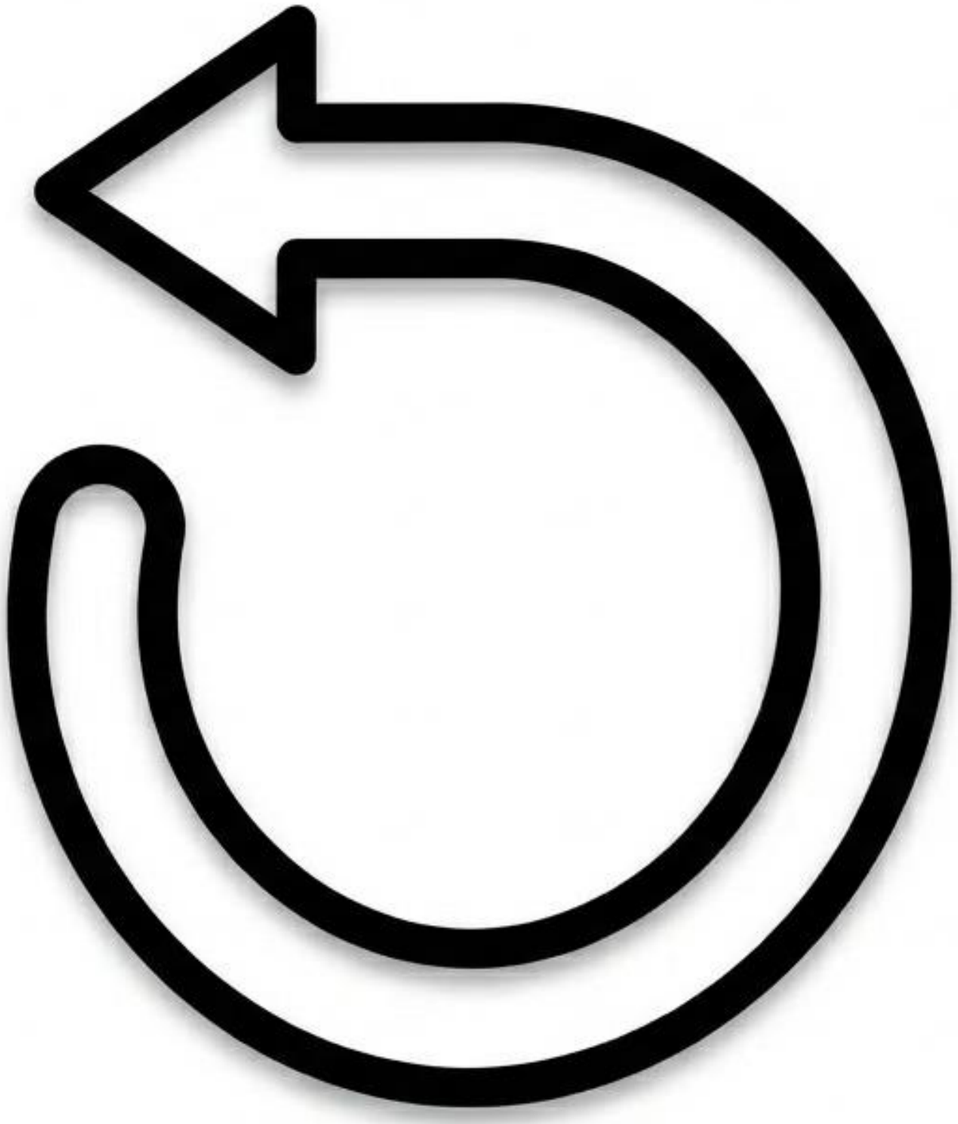
```python
from animal import Animal

class Lion(Animal):
    def eat(self):
        print(self.name, "is eating meat")

class Elephant(Animal):
    def eat(self):
        print(self.name, "is eating plants")
```

Treating different objects uniformly via polymorphism



Polymorphism translates directly to “many forms”. It means that entirely different objects can respond to the exact same method call in completely different ways. This principle allows programs to treat varied objects uniformly, vastly increasing system flexibility.

```python
from animal import Animal

class Lion(Animal):
    def make_sound(self):
        print(self.name, "roars")

class Elephant(Animal):
    def make_sound(self):
        print(self.name, "trumpets")

class Monkey(Animal):
    def make_sound(self):
        print(self.name, "chatters")

class Penguin(Animal):
    def make_sound(self):
        print(self.name, "squawks")
```



Accessing parent methods via the super() function

The **super()** function allows a subclass to securely access methods from its parent class.
It is most commonly used to call the the parent constructor **constructor.**

This mechanism allows developers to efficiently reuse initialisation logic instead of rewriting it from scratch.

```python
from animal import Animal

class Lion(Animal):
    def __init__(self, name, age, pride_size):
        super().__init__(name, age)
        self.pride_size = pride_size

lion = Lion("Simba", 5, 10)

print(lion.name)
print(lion.pride_size)
```

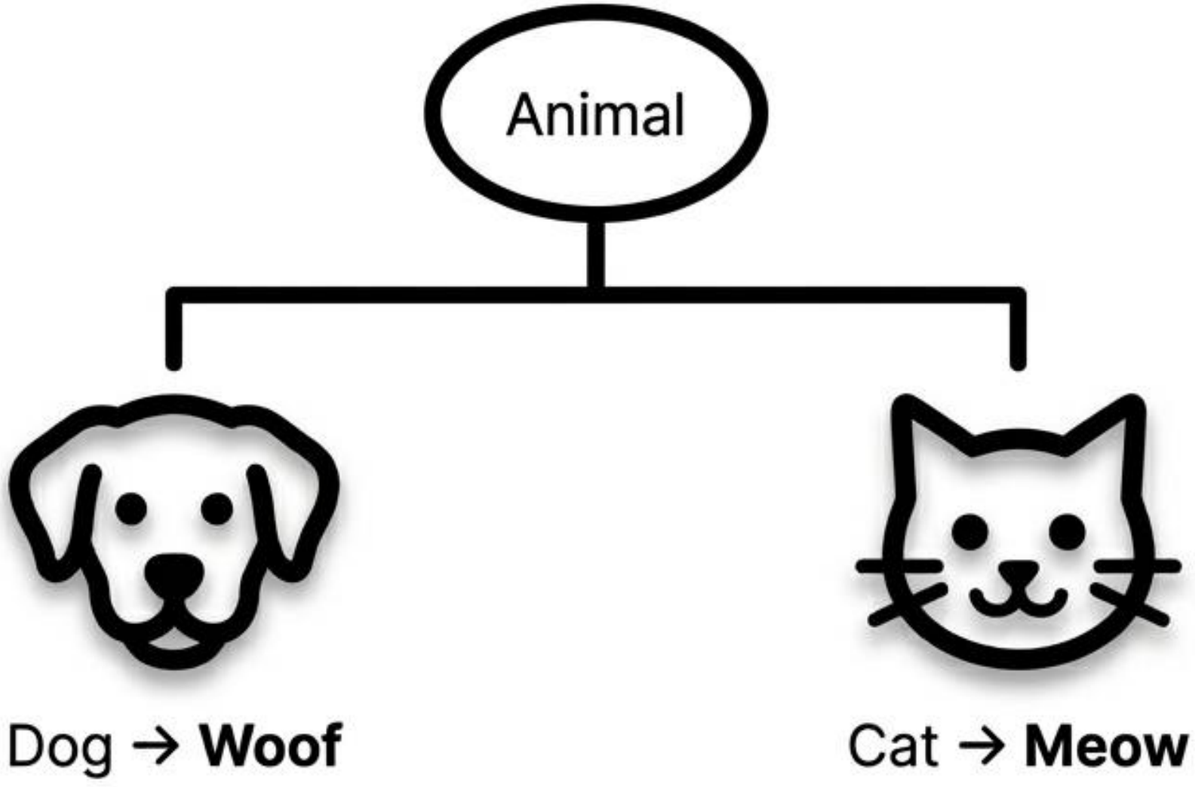
```python
class House:
    def __init__(self, name, bedroom):
        self.name = name
        self.bedroom = bedroom
    def show_details(self):
        print("This is a", self.name)
        print("Bedrooms:", self.bedroom)

class Barrington(House):
    def __init__(self, bedroom, pool):
        super().__init__("Barrington", bedroom)
        self.pool = pool
    def show_details(self):
        super().show_details()
        print("Pool:", self.pool)

class Kensington(House):
    name = "Kensington"
    def __init__(self, bedroom, cinema):
        super().__init__(Kensington.name, bedroom)
        self.cinema = cinema
    def show_details(self):
        super().show_details()
        print("Cinema:", self.cinema)
```

Defining strict contracts with abstract classes

An abstract class represents a broad concept that should not be instantiated directly. Instead, it defines a rigid template or "contract" for subclasses. Abstract classes often contain abstract methods, which must be implemented by any subclass that inherits them.

Forcing implementation across subclasses

Consider an abstract **Animal** class. It defines a **speak()** method, but does not actually implement it. Subclasses must define the specific behaviour to fulfill the contract:



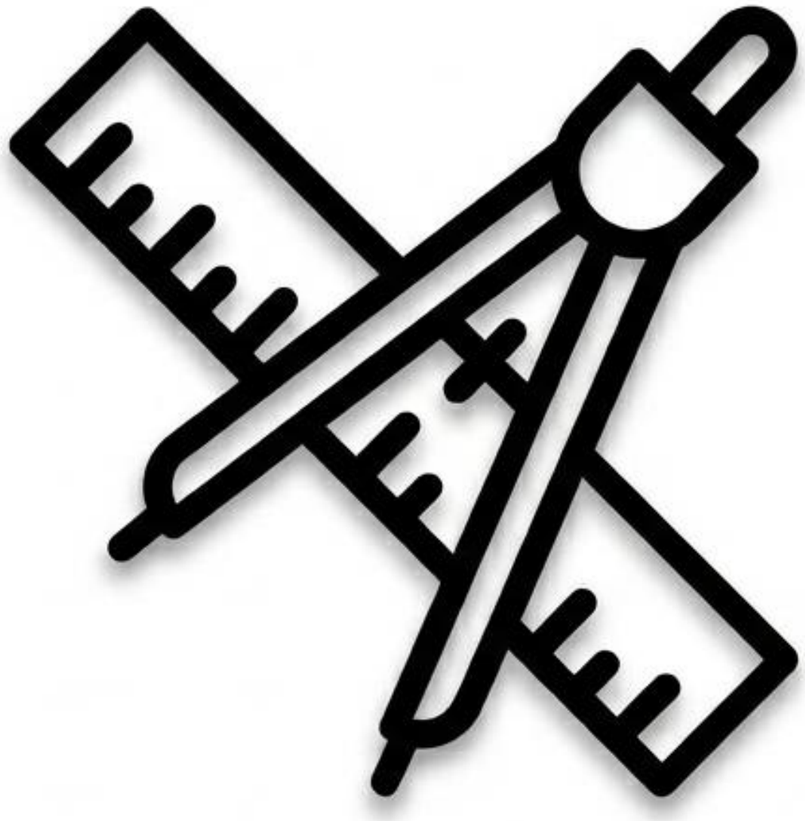
This guarantees every subclass provides the required functionality.

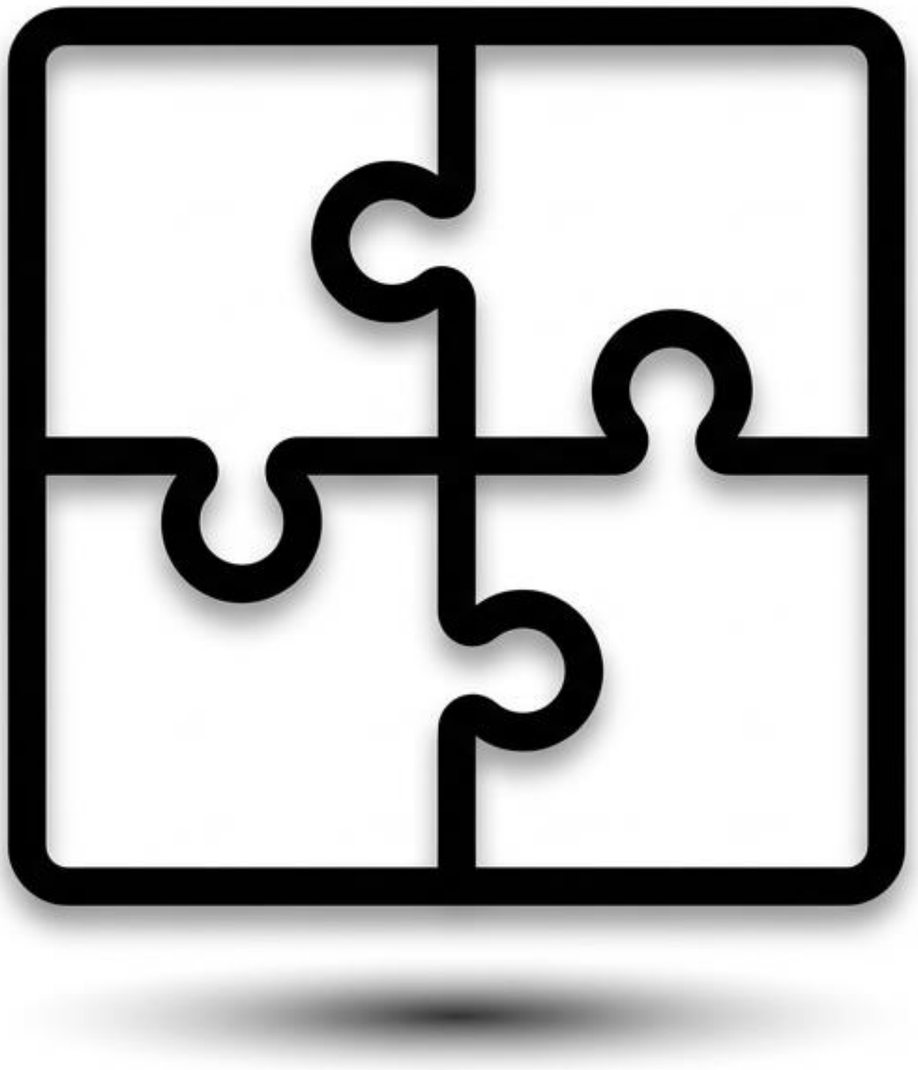
```python
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name, age):
        self.name = name
        self.age = age
    @abstractmethod
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        print(self.name, "barks")

class Cat(Animal):
    def speak(self):
        print(self.name, "meows")

dog = Dog("Buddy", 3)
cat = Cat("Whiskers", 2)

dog.speak()
cat.speak()
```

Enforcing consistency and system reliability

Abstract classes enforce design consistency. By ensuring all subclasses follow the exact same structure, they:

- Prevent incomplete class designs
- Define required behaviours for all subclasses
- Improve overall system reliability
- Encourage clear architectural planning for frameworks





Integrating concepts into modular systems

Modern OOP systems combine these ideas seamlessly:

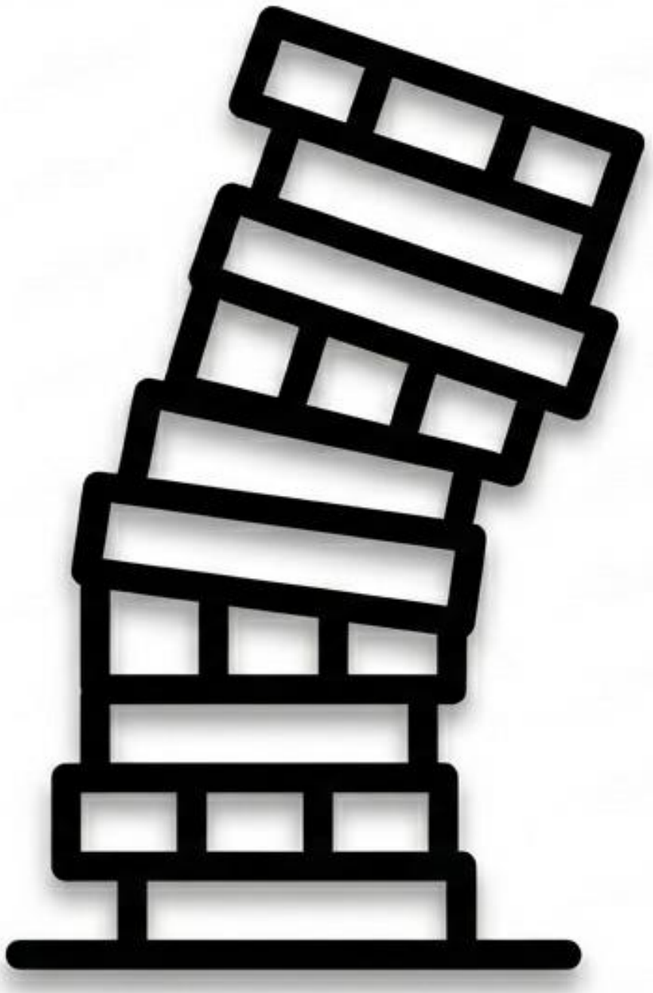
- Inheritance defines the structure.
- Abstract classes define the required behaviour.
- Overriding customises the functionality.
- Polymorphism allows flexible interaction.

Together, they allow developers to design systems that are extensible, maintainable, modular, and easy to understand.

The fragility of inheritance abuse

Inheritance is powerful, but easily misused. It creates fragile systems when:

- Subclasses depend too heavily on parent class behaviour.
- Small changes in the parent class break many child classes.
- The inheritance hierarchy becomes far too deep.
- Classes inherit behaviour they simply do not need.



```python
from animal import Animal

class Bird(Animal):
    def __init__(self, name, age, wingspan):
        super().__init__(name, age)
        self.wingspan = wingspan

    def fly(self):
        print(self.name, "is flying with a wingspan of", self.wingspan, "cm")

class Parrot(Bird):
    def make_sound(self):
        print(self.name, "says Hello!")

class Sparrow(Bird):
    def make_sound(self):
        print(self.name, "chirps")

class Penguin(Bird):
    def make_sound(self):
        print(self.name, "squawks")
```

Logical contradictions in structural design

The Penguin Problem. If a general Bird class defines a fly() method, and a Penguin class inherits from Bird, the relationship breaks. Penguins cannot fly, making the inheritance relationship fundamentally incorrect. Poor inheritance design ultimately creates severe logical contradictions in the codebase.

Composition vs inheritance

Understanding the difference is critical in good software design.

- **Inheritance** describes an **"is-a" relationship** (e.g., a **Dog** is an **Animal**, an **ElectricCar** is a **Car**).
- **Composition** describes a **"has-a" relationship** (e.g., a **Car** has an **Engine**, a **Team** has many **Players**).

While both are important, composition often leads to significantly more maintainable systems.

Modelling relationships to manage complexity

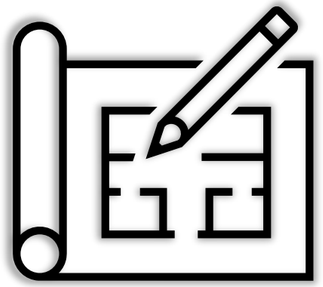
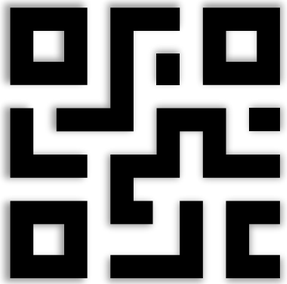
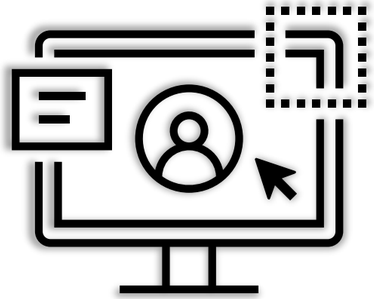
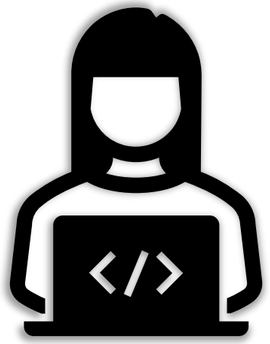
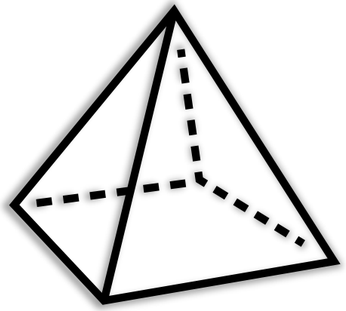
Good OOP design is not about using every feature of the language; it is about modelling relationships correctly.

- Is this object a specialised version of another? → Use Inheritance
- Does this object contain other objects? → Use Composition

Rather than writing rigid code, developers design systems of interacting objects. This yields clearer architecture, highly reusable components, and easily extended systems.

Insights

Final thoughts on OOP?

OOP is about modelling real-world entities and their relationships through classes and objects, aiming for high cohesion and low coupling.

Good design focuses on creating clear, reusable, and maintainable systems.